

To pass strict financial audits, ensure compliance under the UK Merged R&D Scheme, and give your developers an exact blueprint, you must structure the **2-Second Transaction Rule** into a deterministic sequence.

This architecture must explicitly separate the **Synchronous Authorization Loop** (which handles the network timeout) from the **Asynchronous Settlement Loop** (which handles the actual money movement and loyalty updates).

Here is the structured technical architecture for the GoOuts transaction engine.

The 2-Second Time Budget Allocation

When the virtual card is tapped at a commercial terminal, the entire loop must execute within **2,000 milliseconds**. Your server engine slices that budget into precise micro-windows:

[0ms] Card Tap → [300ms] Stripe Webhook Request → [700ms] Firebase Balance Logic → [900ms] Response Dispatched → [2000ms] Network Close

1. **0ms – 300ms:** Card tap at terminal → Mastercard/Visa routing → Stripe processes the authorization payload → Stripe fires a secure webhook request to your serverless endpoint.
2. **300ms – 700ms (The GoOuts Core — 400ms Window):** Your engine calculates multi-pillar balances, checks the fraud/VPC rules, and commits an atomic memory state decision.
3. **700ms – 900ms:** Response payload sent back to Stripe → Stripe commands the network to approve.
4. **900ms – 2000ms:** Safety buffer for network jitter, cell tower latency, and terminal processing.

Step-by-Step Technical Execution Flow

The architecture operates as a two-stage rocket.

Stage 1: The Synchronous Authorization Loop (Time-Critical)

This stage isolates data into lightning-fast memory to issue an immediate "Approve" or "Decline" signal before the network times out.

1. **Payload Ingestion:** Stripe hits your endpoint with an `issuing_authorization.request` payload containing the `merchant_data` (category, cost) and `user_id`.
2. **Fast Cache Read:** The Cloud Function reads from **Firestore Realtime Database (In-Memory)** or a Redis cache cluster—*never* from cold, disk-bound NoSQL tracking logs. This provides sub-5ms read speeds.
3. **The Multi-Pillar Equation Validation:** The engine computes the maximum spendable limit across three pre-allocated pools:

$$\text{\$}\text{\texttt{\text{Total Spendable Power}}} = P_1 (\text{\texttt{Settled Cashback}}) + P_2 (\text{\texttt{Deposited Cash}}) + P_3 (\text{\texttt{Platform Bonus Float}})\text{\$}$$

4. **The Gap Allocation Decision (\$P_4\$):** * If $\text{Transaction Cost} \leq (P_1 + P_2 + P_3)$, the equation balances.
 - o If $\text{Transaction Cost} > (P_1 + P_2 + P_3)$, the engine instantly identifies the missing margin:

$$\text{Gap Amount } (P_4) = \text{Transaction Cost} - (P_1 + P_2 + P_3)$$

- o To keep the user's card from declining right at the counter, your engine relies on your platform's operational float to instantly cover the \$P_4\$ gap.
5. **Cryptographic Handshake:** The Cloud Function immediately responds to Stripe with a JSON body: `{"approve": true}`. The terminal registers success, prints the receipt, and the user walks away happy. **Total Time Elapsing on Your Servers: ~120ms.**

Stage 2: The Asynchronous Settlement Loop (Background Execution)

Once the transaction is safely authorized, the engine spins up an asynchronous "fire-and-forget" pipeline via Google Cloud Tasks or Pub/Sub queues to perform the actual banking math.

1. **Trigger Open Banking VRP Webhook:** If a \$P_4\$ Gap occurred, your background worker initiates an internal Variable Recurring Payment (VRP) call to your open banking gateway (e.g., TrueLayer/Barclays Open Banking API). This pulls the exact \$P_4\$ fiat value from the user's linked personal high-street checking account to reimburse your platform's operational float.
2. **Ledger State Settlement:** The engine locks the user's specific database row and executes a database transaction to update the primary sources of truth:
 - o Deducts 100% of the available \$P_1\$ and \$P_3\$ balances.
 - o Deducts the remaining necessary portion from the real-cash \$P_2\$ wallet.
3. **B2B Commission Intercept:** The system reads the merchant's contract rules (e.g., 20% marketing fee) and dynamically logs a pending automated payout claim from the merchant's connected merchant account.
4. **Generate Post-Purchase Rewards:** The system applies your dynamic cashback calculation algorithms to the transaction value, writing a new pending reward entry to the user's \$P_1\$ (Settled Cashback) matrix.
5. **Push Notification & Optimistic UI Flash:** The system sends a WebSocket payload to the user's device. The **Live 4-Pillar Tracker** in the app UI refreshes automatically to mirror the updated balances, accompanied by a post-purchase confetti or receipt screen.